Problemi dello zaino, Grafi

Materia	Algoritmi
Anno	Secondo Anno
Data	@December 12, 2024
Studiato	Done

Problema Knapsack intero (o $\{0, 1\}$-Knapsack)

Il problema dello zaino nella sua versione intera prevede semplicemente la scelta di un elemento o meno.

> $\{0, 1\}$-*Knapsack gode della proprietà di sottostruttura ottima.*

Allora sia $\{x_1^*, x_2^*, ..., x_n^*\}$ una soluzione ottima al problema:

- $\sum_{i=1}^{n} x_i^* \cdot w_i \leq W$;
- $\sum_{i=1}^{n} x_i^* \cdot v_i$ è massimo.

Consideriamo un sottoproblema. Tale sottoproblema avrà variabili $x_1, ..., x_{j-1}, x_{j+1}, ..., x_n$. Il limite di peso $W' = W - x_j^* \cdot w_j$. La restrizione della soluzione ottima considerata al sottoproblema è $\{x_1^*, ..., x_{j-1}^*, x_{j+1}^*, ..., x_n^*\}$. Supponiamo quindi che tale soluzione non sia ottima. Significa che esiste una soluzione $\{x_1', ..., x_{j-1}', x_{j+1}', ..., x_n'\}$ tale che:

- $\sum_{i=1}^{n} x_i' \cdot w_i \leq W'$;
- $\sum_{i=1}^{n} x_i' \cdot v_i > \sum_{i=1}^{n} x_i^* v_i$.

Allora possiamo prendere la soluzione $\{x_1', ..., x_{j-1}', x_j^*, x_{j+1}', ..., x_n'\}$. Avremo quindi:

- $\sum_{i=1, i\neq j}^{n} x_i' \cdot w_i + x_j^* \cdot w_j \leq W - x_j^* \cdot w_j + x_j^* \cdot w_j = W$;
- $\sum_{i=1, i\neq j}^{n} x_i' \cdot v_i + x_j^* \cdot v_j > \sum_{i=1}^{n} x_i^* \cdot v_i$.

Tuttavia ciò è un assurdo perché contraddice l'ipotesi di soluzione ottima.

Notiamo che in questo caso **non abbiamo utilizzato l'ipotesi che x_i sia un valore binario.** Questo mostra che possiamo ripetere lo stesso procedimento e provare che vale anche la proposizione:

> *Fractional-Knapsack gode della proprietà di sottostruttura ottima.*

Per entrambi i problemi si può utilizzare la programmazione dinamica.

Scelta greedy

Possiamo scegliere gli oggetti tali che il rapporto valore-peso è il più alto. Di conseguenza, possiamo ordinare gli oggetti in modo tale che $k_1 = \frac{v_1}{w_1} > k_2 = \frac{v_2}{w_2}$... e la complessità del problema si riduce a $O(n \log n)$. Quindi vale la proposizione:

> *Fractional-Knapsack gode della proprietà di scelta greedy.*

Per dimostrare la proprietà di scelta greedy basta dimostrare che esiste una soluzione ottima che contiene almeno una scelta greedy.

Supponiamo $k_1 \geq k_2 \geq ... \geq k_n$. Alla prima iterazione, avremo la scelta dell'oggetto k_1.
Il problema diventa:

$$\max \sum_{i=1}^{n} x_i \cdot v_i = \max \sum_{i=1}^{n} q_i \cdot k_i$$

dove $q_i = x_i \cdot w_i$, ossia la frazione di peso presa.

Allora sia $\{ q_1, ..., q_n \}$ una soluzione ottima che non contiene la scelta greedy. Supponiamo che $q_1 = 0$, ossia non stiamo prendendo l'oggetto k_1. Quindi:

$$\sum_{i=1}^{n} q_i \leq W$$

Sia $q_2 \neq 0$. Allora possiamo sostituire l'oggetto 2 con l'oggetto 1 e ottenere l'insieme:

$$\{ q_1^* = q_2, q_2^* = 0, q_3^* = q_3, ..., q_n^* = q_n \}$$

Questo insieme contiene la scelta greedy, in quanto $q_1^* \neq 0$. Inoltre, è una soluzione:

$$\sum_{i=1}^{n} q_i^* = q_1^* + q_2^* + \sum_{i=3}^{n} q_i^* = q_2 + 0 + \sum_{i=3}^{n} q_i = \sum_{i=1}^{n} q_i \leq W$$

Dimostriamo che è ottima:

$$\sum_{i=1}^{n} q_i^* \cdot k_i = q_1^* \cdot k_1 + q_2^* \cdot k_2 + \sum_{i=3}^{n} q_i^* \cdot k_i =$$
$$= q_2 \cdot k_1 + 0 + \sum_{i=3}^{n} q_i \cdot k_i \geq q_2 \cdot k_2 + \sum_{i=3}^{n} q_i \cdot k_i$$

Quindi $\sum_{i=1}^{n} q_i^* \cdot k_i$ è massimo e abbiamo trovato una soluzione ottima che contiene la scelta greedy.

Tuttavia, $\{ 0, 1 \}$-Knapsack **non gode della proprietà di scelta greedy**. Basta mostrare il controesempio.
Prendiamo 3 oggetti:

- il primo ha valore 60 e peso 10;
- il secondo ha valore 100 e peso 20;
- il terzo ha valore 120 e peso 30.

Supponiamo $W = 50$. Chiaramente la soluzione migliore è prendere gli ultimi due oggetti, di valore 220. Ma se operiamo col greedy, notiamo che:

$$k_1 = 6, k_2 = 5, k_3 = 4$$

E la scelta greedy porterebbe ad una soluzione non ottima (primo e secondo oggetto).

Grafi

Esplorazione di un grafo

Sia $G = (V, E)$. Possiamo esplorare il grafo in due modi diversi:

- in profondità;
- in ampiezza.

Ricerca in profondità (Depth First Search)

La **ricerca in profondità** di un grafo, o **Depth First Search (DFS)**, esplora gli archi che dipartono dal vertice v scoperto più recentemente e che sono ancora inesplorati. Una volta che tutti gli archi uscenti da v sono stati esplorati, la ricerca fa retromarcia.

Il processo continua finché tutti i vertici raggiungibili dal vertice sorgente sono stati esplorati. Se nel grafo ci sono ancora dei vertici non esplorati, la ricerca DFS seleziona un nuovo vertice sorgente e ripete la procedura.

Si crea un grafo dei predecessori, ossia una foresta che contiene uno o più alberi DFS. In particolare, si avrà $G_\pi = (V, E_\pi)$ con $E_\pi = \{ (v.\pi, v) : v \in V \land v.\pi \neq \text{null} \}$, dove $v.\pi$ è il predecessore di v nell'esplorazione DFS.

Nella foresta DFS G_π sottografo di G non sono presenti obbligatoriamente tutti gli archi. In particolare:

- gli archi che rimangono in G_π sono rami degli alberi della foresta;
- gli archi che collegano due nodi appartenenti allo stesso albero ma non adiacenti sono detti **archi in avanti**;
- gli archi che collegano nodi di due alberi diversi sono detti **archi incrociati**;
- gli archi che collegano due nodi appartenenti allo stesso albero ma non adiacenti e **in direzione opposta rispetto all'orientamento del cammino nell'albero** sono detti **archi all'indietro**.